

嵌入式智能系统-实验 2

计算机 2303 尤颢辰

2026 年 5 月 15 日

1 问题陈述

基于给出的代码框架，增加、修改其功能。将其改为中断方式，实现 UART0 接收和发送。需要满足：
采用 loopback 模式，发送一个字符串，并接收。发送在复位程序最后进行；
发送和接收都采用中断方式；
编写完整程序，包括中断向量表、复位程序、IRQ 中断服务程序等。全汇编或混合编程均可。

2 代码框架分析

首先给出的代码框架可以分为两部分，分别是 2410addr.h 与 Uart_pool，其中前者是寄存器地址表，是一个纯头文件，不含任何函数，只有 define 宏。它能把 S3C2440 芯片上各个外设的物理内存地址映射成可读的名字。后者是代码的模板，其实现了 UART0 的轮询模式收发数据，简单来说就是 CPU 在发送数据的时候一直等到 TX 缓冲区空再发送，接受时一直等到 RX 有数据再接收，等到了再继续执行，整个过程 CPU 的资源一直被占用。我们修改的目的就是让 UART 能够等到缓冲区有数据的时候主动触发中断，叫 CPU 来接收/发送数据。

2.1 用到的宏定义中的寄存器

GPIO H 组：

rGPHCON：UART 通信使用 H 端口的第二功能，需要配置端口；

rGPHUP：关闭 GPH2/GPH3 内部上拉电阻；

UART 0：

rULCON0：线路控制（决定数据传输格式）

rUCON0：控制寄存器，设置时钟源、TX/RX 工作模式、loopback 使能

rUFCON0：FIFO 控制；

rUMCON0：Modem 控制；

rUTRSTAT0：错误状态寄存器（发送）；

rUERSTAT0：错误状态寄存器（接收）；

rUBRDIV0：波特率分频值；

RdURXH0：读取接收缓冲区（取一个字节）；

WrUTXH0：写入发送缓冲区（发一个字节）；

中断控制器:

rINTMOD: 中断模式寄存器: 写 0 设为 IRQ 模式;

rINTMSK: 主中断屏蔽;

rINTSUBMSK: 子中断屏蔽;

rSUBSRCPND: 子中断源挂起,ISR 中判断是 RX 还是 TX 触发;

rSRCPND: 中断源挂起,ISR 结束时写 1 清除 UART0 位;

rINTPND: 中断挂起,ISR 结束时写 1 清除 UART0 位。

2.2 源代码框架的实现

首先, **Uart_Init** 实现了初始化。其主要做了下面几件事:

1. 将 GPIO 对应引脚切换为串口功能, 同时设置开启内部上拉电阻, 避免误触发。
2. 禁用 FIFO 缓冲队列, 采用单字节收发模式。同时关闭自动流控 (AFC) 功能, 简化收发逻辑。
3. 配置通信格式: 8 位数据位、1 位停止位、无奇偶校验、普通传输模式。
4. 配置工作模式: 目前是设置为轮询模式, 在后面的修改之中需要处理。
5. 计算并设置波特率, 把结果写入相应寄存器。

其次, 数据发送由 **Uart_SendByte** 与 **Uart_SendString** 两个函数实现, 主要实现下面的功能:

Uart_SendByte 负责发送单个字节, 函数首先轮询查询 UTRSTAT0, 看看缓冲区有没有未发送的数据。若不能写入, 轮询等待; 若能写入, 短暂延时之后, 通过 **WrUTXH0(data)** 将数据写入发送缓冲寄存器。

然后是数据接收部分, 由 **Uart_Getch** 与 **Uart_GetString** 两个函数实现, 主要实现:

Uart_Getch 负责接收单个字节, 函数轮询查询 UTRSTAT0, 若不能接收数据, 轮询等待; 若能接收数据, 检查 UERSTAT0, 看有没有错误导致接受失败。最后通过 **RdURXH0** 从接收缓冲寄存器读出数据并返回。

最后是数据解析部分, 由 **Uart_GetIntNum** 函数实现, 主要实现了在接收字符串的基础之上, 将其解析为整数返回, 可以返回为十进制, 十六进制和负数三种形式。

3 在源代码框架基础上进行修改

我们总结一下需要修改/增加几个部分: 首先初始化函数需要修改, 要把 TX/RX 从轮询模式改为中断模式, 并开启 loopback, 此外还要新增中断控制器的初始化。其次还要新增中断服务程序, 这个程序能够读取接收到的字符, 能够回送发出字符, 还能清除中断标志。最后要新加一个 main 函数, 处理一系列工作。下面对这三个部分依次修改:

3.1 修改 Uart_Init

主要任务就是开启 loopback 使能, 开启 RX/TX 的中断模式, 再加上中断控制器的配置。

代码如下:

```
1 void Uart_Init_IRQ(int pclk, int baud)
2 {
3     rGPHCON &= ~0xF0;
4     rGPHCON |= 0xa0;    /* GPH2->TXD0, GPH3->RXD0 */
5     rGPHUP    = 0x0;
```

```

6   if (pclk == 0) pclk = PCLK;
7   rUFCON0 = 0x0;      /* 关闭 FIFO */
8   rUMCON0 = 0x0;      /* 关闭 AFC */
9   rSUBSRCPND |= (BIT_SUB_TXD0 | BIT_SUB_RXD0);      /* 8位数据, 1停止位, 无校验 */
10  rUCON0 = 0x25;
11  rUBRDIV0 = ((int)(pclk / 16 / baud + 0.5) - 1);
12  /* 清除三个挂起寄存器, 防止残留中断 */
13  rSUBSRCPND |= 0x3;      /* 清 SUBSRCPND[1:0] (TXD0、RXD0) */
14  rSRCPND |= BIT_UART0;   /* 清 SRCPND[28] */
15  rINTPND |= BIT_UART0;   /* 清 INTPND[28] */
16  /* 打开总中断屏蔽和收发子中断屏蔽 */
17  rINTMOD = 0x0;
18  rINTMSK &= ~BIT_UART0;
19  rINTSUBMSK &= ~(BIT_SUB_TXD0 | BIT_SUB_RXD0);
20 }

```

3.2 增加中断处理函数

我们编写 UART0_ISR 作为中断处理函数, 并配套编写 Uart_RX_Handler 和 Uart_TX_Handler 两个子程序, 分别处理接收和发送中断。主 ISR 进入时先屏蔽收发中断防止干扰, 根据 rUTRSTAT0 状态分别调用对应子程序, 完成后清除挂起寄存器并取消屏蔽等待下一次中断。具体代码实现如下:

```

1  static void Uart_RX_Handler(void)
2  {
3      char c = RdURXH0;    /* 读出接收缓冲区字符 */
4      (void)c;             /* loopback 下收到的是自己发的, 不需要额外处理 */
5  }
6  static void Uart_TX_Handler(void)
7  {
8      if (tx_buf != NULL && tx_idx < tx_total) {
9          WrUTXH0(tx_buf[tx_idx++]);    /* 发下一个字符 */
10     } else {
11         rINTSUBMSK |= BIT_SUB_TXD0;    /* 发完, 关闭 TX 子中断 */
12     }
13 }
14 void __irq IRQ_Handler(void)
15 {
16     typedef void (*isr_t)(void);
17     /* 软件向量表基址 0x33FFFF24, 每项 4 字节, INTOFFSET 给出中断号 */
18     isr_t isr = (isr_t)((volatile unsigned *) (0x33FFFF24 + rINTOFFSET * 4));
19     isr();
20 }
21 void UART0_ISR(void)
22 {
23     rINTSUBMSK |= (BIT_SUB_TXD0 | BIT_SUB_RXD0); // 屏蔽收发中断
24     // 根据 UTRSTAT 判断并调用对应子程序
25     if (rUTRSTAT0 & 0x1) Uart_RX_Handler();
26     if (rUTRSTAT0 & 0x2) Uart_TX_Handler();
27     /* 清除挂起寄存器, 顺序: SUBSRCPND -> SRCPND -> INTPND */
28     rSUBSRCPND |= (BIT_SUB_TXD0 | BIT_SUB_RXD0);
29     rSRCPND |= BIT_UART0;
30     rINTPND |= BIT_UART0;

```

```

31  /*取消中断屏蔽，等待下一次中断 */
32  rINTSUBMSK &= ~BIT_SUB_RXD0;
33  if (tx_buf != NULL && tx_idx < tx_total)
34      rINTSUBMSK &= ~BIT_SUB_TXD0;
35  }

```

该中断处理函数的整体逻辑如下：每当 UART0 发生收发事件时，CPU 跳入 IRQ_Handler，通过读取 INTOFFSET 寄存器从软件向量表中取出 UART0_ISR 的地址并调用。UART0_ISR 首先屏蔽收发中断防止嵌套干扰，随后通过查询 rUTRSTAT0 判断是接收还是发送触发，之后分别调用 Uart_RX_Handler 读走 loopback 回来的字符，或用 Uart_TX_Handler 发出下一个待发字符，处理完成后清除三个挂起寄存器，最后根据是否还有待发数据决定是否重新打开 TXD0 中断，避免字符串发完后 TX 中断反复触发，整个字符串的发送就是靠这样一次次中断驱动、逐字符滚动完成的。

3.3 完善主函数

我们在 main 函数中按顺序串联所有初始化步骤，完成从硬件配置到中断使能的全部准备工作，最终进入空转等待中断驱动后续收发。具体代码实现如下：

```

1  int main(void)
2  {
3      Uart_Init_IRQ(0, 115200);
4      pISR_UART0 = (unsigned)UART0_ISR;
5      EnableIRQ();
6      Uart_SendString_IRQ("以中断方式\r\n");
7      while (1) { }
8      return 0;
9  }

```

4 编译，测试

我们使用 CodeWarrior IDE 搭建工程，编译测试。

由于我没有找到工程模板，我参考了 ADS 的自带示例，其存放在 C:\Files1.2_irq 路径下。将其启动文件复制至工程目录作为运行时框架。

整个工程包含以下文件：

文件	来源	作用
vectors.s	rps_irq 示例	异常向量表，IRQ 向量指向 IRQ_Handler
init.s	rps_irq 示例	复位处理，初始化各模式栈指针，跳转至 __main
stack.s	rps_irq 示例	定义栈空间
heap.s	rps_irq 示例	定义堆空间
retarget.c	rps_irq 示例	C 库底层适配

文件	来源	作用
uart_irq.c	本次编写	UART0 中断模式主逻辑
2410addr.h	本次编写	S3C2440 寄存器地址宏定义
option.h	本次编写	PCLK 频率与软件向量表入口 pISR_UART0
2410lib.h	本次编写	EnableIRQ() 内联汇编，清 CPSR I 位开放 IRQ

表 1: 工程文件列表

可以看到，还有 2410lib.h 和 option.h 是自己编写的，这两份代码都十分简短，因为我没有找到标准的 S3C2440 ADS 工程模板，只在这两个头文件里实现了本项目必要的功能。代码如下：

2410lib.h:

```

1 #ifndef __2410LIB_H__
2 #define __2410LIB_H__
3
4 __inline void EnableIRQ(void)
5 {
6     int tmp;
7     __asm
8     {
9         MRS tmp, CPSR
10        BIC tmp, tmp, #0x80
11        MSR CPSR_c, tmp
12    }
13 }
14 #endif

```

option.h

```

1 #ifndef __OPTION_H__
2 #define __OPTION_H__
3 #define PCLK    50000000
4 #define pISR_UART0 (*(volatile unsigned *)0x33FFFF94)
5 #endif

```

接下来我们汇编即可，能看到整个工程已经能汇编成功了。

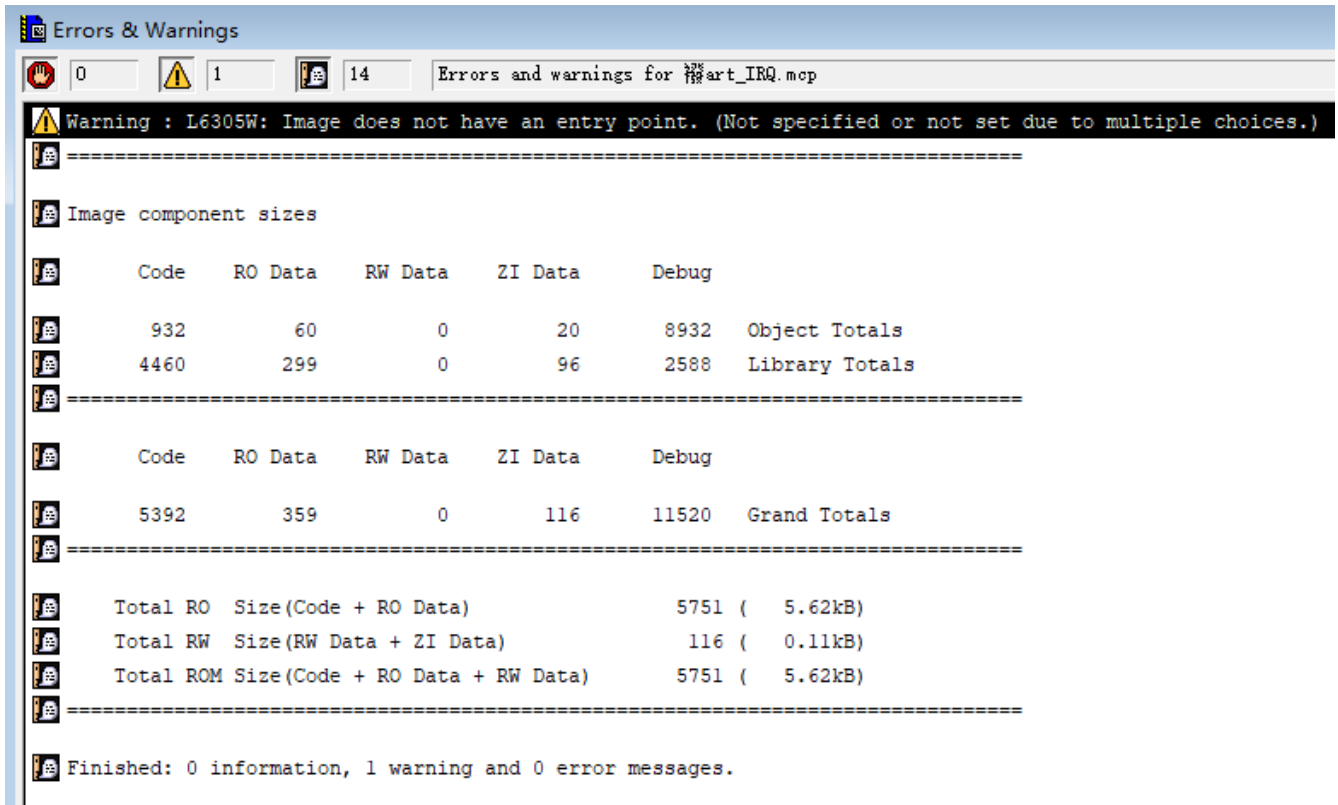


图 1: Make

可以看到最后有一个 Warning，是因为我们没有指定 entry point 导致的。这在真实的硬件中是无所谓的，因为上电后一定是从 0x0 开始取指令执行。想要解决这个问题在 Image entry point 填入 Reset_Handler，重新编译即可。

具体的所有代码，详见打包的附件 code.zip。